VANAR Chain

SMART CONTRACT AUDIT REPORT

IMMUNEBYTES

28 July 2023

Introduction **3**

About VANAR Chain 3

About ImmuneBytes 3

Documentation Details **4**

Audit Goals **4**

Audit Process & Methodology **5**

Audit Details **6**

System Architecture **6**

Security Level References **7**

Severity Status References **8**

Severity Status **8**

Finding **9**

Critical Severity Issues **10**

High Severity Issues **10**

Medium severity issues **13**

Low severity issues **16**

Informational **18**

Test Coverage **21**

Test Result **21**

Gas Report **22**

Concluding Remarks **23**

Disclaimer **23**

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About VANAR Chain

VANAR Chain is a blockchain solution, carefully designed in response to our years of experience working closely with mainstream brands and gaining a deep understanding of their specific needs. VANAR will be an indispensable foundation for brands, gaming companies, and the next generation of XR, providing a fully decentralised platform with EVM compatibility, thus enabling a straightforward transition from Web 2.0 to the decentralized environment of Web 3.0.

Recognizing the common challenges of scalability and cost-effectiveness in the blockchain industry, VANAR Chain is built to deliver high-speed transactions at minimal costs.

Our unique DApp-as-a-Service capabilities, another core feature of VANAR Chain, streamline the process of building and deploying decentralized applications. This component was crafted based on direct insights into the complexities developers often face.

In summary, VANAR Chain stands as a practical, experience-driven solution for the mainstream, aiming to exploit the benefits of decentralized technology in a user-centric and economically efficient way.

Visit https://vanarchain.com/ to know more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting smart contract audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 205+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The team has provided the following doc for audit:

1. Document: VANRY Token - Solution Doc

Audit Goals

The focus of the audit was to verify that the smart contract system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within each contract and within the system of contracts.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established smart contract best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include

    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

ImmuneBytes team has performed thorough testing of the project, starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is a structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors which includes -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Details

Project Name	VANAR Chain
Platform	EVM
Contract	KolectToken.sol, TVKSwap.sol, VANRY.sol
Languages	Solidity
GitHub Link	https://github.com/TerraVANAR ChainCo/VANAR Chain-token/tree/
Commit	1241cd9f78cf132ecca2e51dc98a4d4095775017
Platforms & Tools	Remix IDE, Truffle, VScode, Contract Library, Slither, SmartCheck, Fuzz

System Architecture

The system architecture is divided into two main components: the Token Architecture and the Swap Contract Architecture.

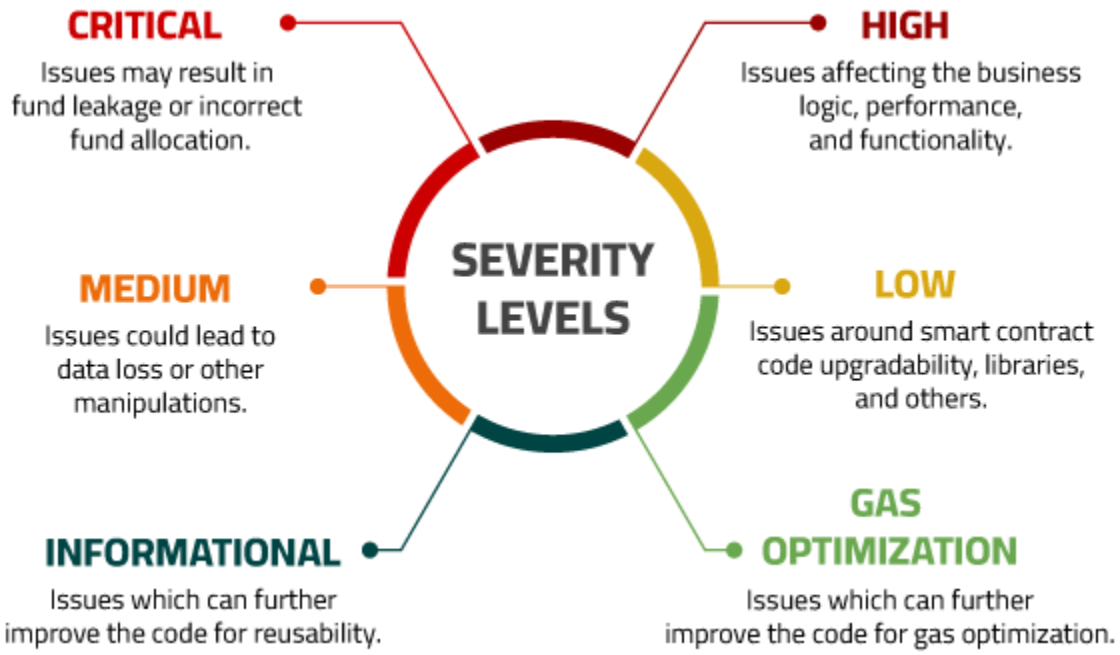
The Token Architecture revolves around the VANRY Token. This token has a maximum supply of 2.4 billion tokens, with an initial supply of 1.2 billion. The token attributes include its contract name, type, symbol, decimals, max supply, initial supply, and whether it's ownable, pausable, mintable, and burnable.

The Swap Contract Architecture focuses on the swap token contract. A multisig wallet must own this contract, allowing the owner to pause/unpause the swap. The contract ensures that the swap is open and not paused, the amount of tokens to be swapped is a positive number, and the swap contract holds enough balance of new tokens to complete a swap successfully. The contract also includes a function to withdraw any ERC20 token balance from the contract. This includes swapped TVK, a new token, and any other token sent to the swap contract address.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:



This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Severity Status References

Severity Status

Issues	Critical	High	Medium	Low	Informational	Gas
Open	-	-	-	-	-	-
Fixed	-	2	1	2	4	-
Redacted	-	-	-	-	-	-
Acknowledged	-	-	2	-	-	-

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Finding

#	Findings	Risk	Status
1.	Reentrancy Attack Vector	High	Fixed
2.	Centralization Risk and Potential Rugpull Indicator	High	Fixed
3.	Arbitrary Token Withdrawal	Medium	Acknowledged
4.	Use of Constants for Token Addresses	Medium	Acknowledged
5.	Lack of Preconditions for Swap Toggle	Medium	Fixed
6.	Event Naming Convention	Low	Fixed
7.	Inefficient Usage of SafeMath	Low	Fixed
8.	Use of Floating Pragma	Informational	Fixed
9.	Inefficient Use of Comparison Operator	Informational	Fixed
10.	Inconsistent Naming Convention for Constants	Informational	Fixed
11.	Insufficient NatSpec Comments	Informational	Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

No issues were found.

High Severity Issues

1. Centralization Risk and Potential Rugpull Indicator
Description: The withdrawToken function can only be called by the contract owner. This introduces a centralization risk as the owner has full control over the funds in the contract. It could also indicate a potential rug pull, where the owner withdraws all the funds and leaves other users with worthless tokens.

Code Affected:
function withdrawToken(
address _beneficiary,
uint256 _amount,
address _tokenAddress
) public onlyOwner {
...
}

Impact: The contract owner has the ability to withdraw all tokens from the contract, which could be exploited in a rugpull scenario. This could lead to significant financial loss for users and damage the project's reputation.
Recommendation: Implement a multisig wallet for administrative functions to reduce centralization risk. This would require multiple parties to agree on transactions before they are executed, reducing the risk of a single party acting maliciously.
Developer's Response: "We can pull back the deposited VANRY tokens out of the contract which we are to deposit to facilitate swap for TVK. Again this is going to be done by us via owner only mutlisig wallet and business won't be doing this for any reason other than the fact that if the business says that after X time we won't allow people to swap their TVK and then only we can withdraw available VANRY tokens (this is very rare case)."
Auditor's Response: Acknowledged with no follow-up required.
Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

2. **Reentrancy Attack Vector**
Description: The withdrawToken function is susceptible to a reentrancy attack. It calls an external contract (via the IERC20(_tokenAddress).transfer(_beneficiary, _amount); line) before it finishes its state changes. This could allow an attacker to recursively call the withdrawToken function before the state changes from the previous call have been completed.
Code Affected: IERC20(_tokenAddress).transfer(_beneficiary, _amount);
Impact: An attacker could potentially drain the contract of its funds by recursively calling the withdrawToken function. This could lead to significant financial loss for the contract and its users.
Recommendation: Implement a mutex or use the Checks-Effects-Interactions pattern to prevent reentrancy attacks. This would involve moving the external call to the end of the function after all state changes have been complete.
Developer's Response: "First of all, this function can only be invoked by the multi-sig owner of the contract. There is no user-generated transaction involved. In our case, there are no additional state changes involved if we see the code, only the event is emitted after this call."
Auditor's Response: The auditor team at ImmuneBytes acknowledges the dev response with a strong disclaimer that other identified risks (e.g. 4, 5, 6, and 8) amplify the risk and, therefore reentrancy essentially become an unnecessary attack vector in this case.
Developer's Response: "So what I understand is all of the risks all linked to the owner-only multisig wallet - if that is compromised, meaning a couple of participating keys, then there is trouble; otherwise, we got no issues. Also, do you mean we are not getting working POC on the reentrancy attack for this, and we can safely say that 'there is no chance of a reentrancy attack in any form? If there is any, then really we need to understand, and we can easily put a reentrancy guard on the swap function."

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Auditor's Response: "To answer the queries: Yes, we have seen in multiple scenarios that a multisigs being vulnerable brings the whole protocol down, yet it is the only safe way to execute ops. So yes, there exists no issue otherwise. In a holistic view, the audit team does not find a proof of concept susceptible to bearing potential loss to the contract due to reentrancy, as the ultimate loss can still be carried in just a single call. The reentrancy attack vector exists in the smart contract; we can not say that it does not exist. However, from a game theory point of view, it is not feasible to execute such an attack, as the ultimate benefit that can be reaped out of the contract is not dependent on reentering the contract; rather, a single malicious call is liable to make the contract absolutely vulnerable.
Auditor's Response: Acknowledged with no follow-up required.
Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Medium severity issues

1. **Arbitrary Token Withdrawal**
Description: The withdrawToken function allows the contract owner to withdraw any arbitrary token from the contract. This could potentially allow the owner to withdraw tokens that are not intended to be withdrawn, disrupting the intended functionality of the contract.
Code Affected: IERC20(_tokenAddress).transfer(_beneficiary, _amount);
Impact: The contract owner could potentially withdraw tokens that are not intended to be withdrawn. This could disrupt the intended functionality of the contract and potentially lead to financial loss for users if the withdrawn tokens were meant to be part of the contract's operations.
Recommendation: Restrict the tokens that can be withdrawn by the contract owner to only the intended tokens. This could be done by checking that _tokenAddress is one of the intended tokens before executing the transfer.
Developer's Response: "We have intentionally generalized this function to withdraw both previous and new Token. Plus this function is restricted to onlyOwner modifier, which means this functionality can only be done by the multi-sig owner wallet"
Auditor's Response: The auditor team at ImmuneBytes believes that, based on empirical evidence, a compromised malicious admin (or a compromised multisig contract in this case) is liable to withdraw any token to their address, essentially using the contract for exploitable cases such as; money laundering, price manipulation by sudden inflation and sell-off, interaction with malicious tokens impact in unexpected ways ultimately violating the users and community trust.
Status: Acknowledged

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

2. Use of Constants for Token Addresses

Description:

The contract uses constants to store the addresses of the legacyTVKToken and newTVKToken. This means that these addresses cannot be changed after the contract is deployed. If one of these tokens becomes non-functional or needs to be replaced, the contract would not be able to accommodate this change.

Code Affected:

```
address public constant legacyTVKToken = 0x41eB97b392A0AD1705F2B1193Cf85832228F8770;
address public constant newTVKToken = 0xf58F702C733A31e7d0525219Aa87E614dacA593b;
```

Impact:

If one of the tokens becomes non-functional or needs to be replaced, the contract would not be able to accommodate this change. This could disrupt the functionality of the contract and potentially lead to financial loss for users.

Recommendation:

Instead of using constants, consider using state variables for the token addresses. This would allow the token addresses to be updated if necessary. Ensure that only a trusted role (such as the contract owner) can update these addresses to prevent unauthorized changes.

Developer Response:

"This is fine in our case as we will know exactly know the token address of the TVK and VANRY token and that will never change. Save Gas fees. Give transparency."

Auditor Response:

The auditor team at IB believes that smart contract deployments are prone to human error, and thus the hardcoded address for the newTVKToken is equally liable to a faulty deployment requiring an update in such case.

Additionally, based on the client's confirmation, it is ensured by the developer team that, in the event mentioned, a new swap contract with the correct token guids will be deployed before enabling the swap process.

Status: **Acknowledged**

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

3. Lack of Preconditions for Swap Toggle

Description:
The swapToggle function does not have any preconditions. This means that the contract owner can enable or disable the token swap functionality at any time without any restrictions. This could potentially be exploited to disrupt the functionality of the contract.

Code Affected:

```
function swapToggle() public onlyOwner {
  if (swapStatus) {
    swapStatus = false;
  } else {
    swapStatus = true;
  }
  emit UpdateSwapStatus(swapStatus);
}
```

Impact:
The contract owner could potentially enable or disable the token swap functionality at will, disrupting the functionality of the contract and potentially leading to financial loss for users.

Recommendation:
Consider adding preconditions to the swapToggle function to restrict when the token swap functionality can be enabled or disabled. This could be based on factors such as the time since the contract was deployed, the total supply of tokens, or other relevant conditions.

Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Low severity issues

1. **Event Naming Convention**
Description: The events in the contract are named as actions (TokenSwap, UpdateSwapStatus, TokenWithdraw) rather than as state changes. This is not in line with the recommended Solidity naming conventions, which suggest that events should be named as past-tense verbs to indicate that an action has been completed.

Proof-of-Concept:
event TokenSwap(address indexed _beneficiary, uint256 _amount);
event UpdateSwapStatus(bool _status);
event TokenWithdraw(address indexed _beneficiary, uint256 _amount, address indexed _tokenAddress);

Impact: While this does not have a direct impact on the functionality or security of the contract, it could lead to confusion for developers or users who are interacting with the contract. Following standard naming conventions can make the contract easier to understand and interact with.
Recommendation: Rename the events to indicate state changes. For example, TokenSwap could be renamed to TokensSwapped, UpdateSwapStatus could be renamed to SwapStatusUpdated, and TokenWithdraw could be renamed to TokensWithdrawn.
Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

2. **Inefficient Usage of SafeMath**
Description: The contract uses the SafeMath library for arithmetic operations. However, as of Solidity version 0.8.x, the language has built-in overflow and underflow protection, making the use of SafeMath redundant and inefficient in terms of gas costs.
Code Affected: using SafeMath for uint256;
Impact: This does not pose a security risk, but it does lead to unnecessary gas costs for contract interactions.
Recommendation: Remove the usage of SafeMath and rely on Solidity's built-in overflow and underflow protection.
Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Informational

1. Use of Floating Pragma
Description: The contract uses a floating pragma statement (pragma solidity ^0.8.0;). This means that the contract could be compiled with any compiler version from 0.8.0 (inclusive) to version 0.9.0 (exclusive). If the contract is compiled with a newer compiler version, it could introduce breaking changes that were not present in the older version, leading to unexpected behavior.
Code Affected: pragma solidity ^0.8.0;
Impact: The use of a floating pragma could lead to the contract behaving unexpectedly or becoming vulnerable to new security issues if it is compiled with a different version of the Solidity compiler. This could potentially compromise the security of the contract and the funds it holds.
Recommendation: Specify a fixed compiler version to ensure the contract's behavior remains consistent and predictable. For example, change pragma solidity ^0.8.0; to pragma solidity 0.8.0;.
Developer Response: "It can be done, but in that case, we have to update the openzeppelin's imported contract pragma as well. We have locked the solidity compiler version (0.8.13) in our truffle-config (to be specific) which makes sure to deploy every contract with that version. After deployment, proper testing has been done as well."
Auditor Response: Acknowledged with no follow-up required.
Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

2. **Inefficient Use of Comparison Operator**
Description: The contract uses the > operator for comparison in the require statements of the swapToken and withdrawToken functions. This operator is less gas efficient than the != operator.
Code Affected: require(_amount > 0, "Swap Contract: Invalid amount!");
Impact: Using the > operator could lead to slightly higher gas costs for contract interactions. While the impact is likely minimal, it could add up over a large number of transactions, making the contract less attractive to users due to higher transaction costs.

Recommendation:
Use the != operator for comparison to reduce gas costs.
For example, change
require(_amount > 0, "Swap Contract: Invalid amount!");
to
require(_amount != 0, "Swap Contract: Invalid amount!");

Status: Fixed

3. **Inconsistent Naming Convention for Constants**
Description: The constants in the contract (tokenName, tokenSymbol, maxSupply, initialSupply) are not in uppercase. This is not in line with the recommended Solidity naming conventions, which suggest that constants should be named in uppercase.

Code Affected:
string public constant tokenName = "New Token";
string public constant tokenSymbol = "NT";
uint256 public constant maxSupply = 1700000000;
uint256 public constant initialSupply = 1200000000;

Impact: While this does not have a direct impact on the functionality or security of the contract, it could lead to confusion for developers or users who are interacting with the contract. Following standard naming conventions can make the contract easier to understand and interact with.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Recommendation:
Rename the constants to be in uppercase. For example, tokenName could be renamed to TOKEN_NAME, tokenSymbol could be renamed to TOKEN_SYMBOL, maxSupply could be renamed to MAX_SUPPLY, and initialSupply could be renamed to INITIAL_SUPPLY.

Status: Fixed

4. Insufficient NatSpec Comments

Description:

The mint function does not have NatSpec comments explaining its functionality. NatSpec comments are important for understanding the purpose and functionality of functions, especially for public and external functions that can be called by users or other contracts.

Code Affected:

```
function mint(address _account, uint256 _amount, string memory _purpose) public {
   super.mint(_account,_amount);
   emit MintToken(_account, _amount, _purpose);
}
function _mint(
   address account,
   uint256 amount
) internal VANAR ChainI override(ERC20, ERC20Capped) {
   require(totalSupply() + amount <= cap(), "ERC20Capped: cap exceeded");
   super._mint(account, amount);
}
```

Impact:
The lack of NatSpec comments could lead to confusion for developers or users who are interacting with the contract. This could potentially result in incorrect usage of the function.

Recommendation:
Add NatSpec comments to the mint functions to explain its functionality. The comments should explain the purpose of the function, its parameters, its return values (if any), and any events it emits.

Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Test Coverage

```
Analysing contracts...
Running tests...
| File                        | % Lines        | % Statements   | % Branches     | % Funcs        |
|-----------------------------|----------------|----------------|----------------|----------------|
| contracts/KolectToken.sol   | 100.00% (3/3)  | 100.00% (3/3)  | 100.00% (2/2)  | 100.00% (2/2)  |
| contracts/MockERC20.sol     | 70.83% (17/24) | 68.00% (17/25) | 40.00% (4/10)  | 60.00% (3/5)   |
| contracts/MockTVKSwap.sol   | 82.35% (14/17) | 81.25% (13/16) | 60.00% (6/10)  | 83.33% (5/6)   |
| contracts/TVKSwap.sol       | 0.00% (0/15)   | 0.00% (0/14)   | 0.00% (0/10)   | 0.00% (0/5)    |
| contracts/VirtuaBinary.sol  | 100.00% (4/4)  | 100.00% (4/4)  | 100.00% (0/0)  | 100.00% (3/3)  |
| script/Counter.s.sol        | 0.00% (0/1)    | 0.00% (0/1)    | 100.00% (0/0)  | 0.00% (0/2)    |
| Total                       | 59.38% (38/64) | 58.73% (37/63) | 37.50% (12/32) | 56.52% (13/23) |
```

Test Result

```
[⠊] Compiling...
No files changed, compilation skipped

Running 10 tests for test/TVKSwap.t.sol:TVKSwapTest
[PASS] testAddress() (gas: 5228)
[PASS] testBalanceOfLegacyToken() (gas: 16197)
[PASS] testBalanceOfNewToken() (gas: 16164)
[PASS] testInitialSwapStatus() (gas: 7608)
[PASS] testSwapToggleByNonOwnerShouldFail() (gas: 12748)
[PASS] testSwapToggleByOwner() (gas: 17729)
[PASS] testSwapTokenWhenDisabledShouldFail() (gas: 12902)
[PASS] testSwapTokenWhenEnabled() (gas: 100689)
[PASS] testWithdrawTokenByNonOwnerShouldFail() (gas: 17280)
[PASS] testWithdrawTokenByOwner() (gas: 52071)
Test result: ok. 10 passed; 0 failed; finished in 2.34ms

Running 14 tests for test/KolectToken.t.sol:KolectTokenTest
[PASS] testAddressOfToken() (gas: 5184)
[PASS] testBurn() (gas: 27371)
[PASS] testBurningMoreThanBalanceShouldFail() (gas: 15296)
[PASS] testInitialBalance() (gas: 9984)
[PASS] testMintingBeyondCapShouldFail() (gas: 12514)
[PASS] testMintingByNonOwnerShouldFail() (gas: 15309)
[PASS] testMintingWithinCap() (gas: 27149)
[PASS] testPauseAndTransfer() (gas: 34528)
[PASS] testPausingByNonOwnerShouldFail() (gas: 13005)
[PASS] testTransfer() (gas: 45675)
[PASS] testTransferFrom() (gas: 58276)
[PASS] testTransferToZeroAddressShouldFail() (gas: 10856)
[PASS] testUnpauseAndTransfer() (gas: 53834)
[PASS] testUnpausingByNonOwnerShouldFail() (gas: 39414)
Test result: ok. 14 passed; 0 failed; finished in 1.92ms

Running 13 tests for test/VirtualBinary.t.sol:NewTokenTest
[PASS] testAddress() (gas: 5184)
[PASS] testApproveAndTransferMoreThanApproved() (gas: 40864)
[PASS] testBurn() (gas: 27328)
[PASS] testBurnFrom() (gas: 39809)
[PASS] testBurningMoreThanBalance() (gas: 16528)
[PASS] testInitialSupply() (gas: 14658)
[PASS] testMintingBeyondCapShouldFail() (gas: 17957)
[PASS] testMintingByOwner() (gas: 49230)
[PASS] testMintingWithEmptyPurpose() (gas: 48773)
[PASS] testPauseAndTransfer() (gas: 37444)
[PASS] testTransfer() (gas: 45663)
[PASS] testTransferMoreThanBalance() (gas: 18667)
[PASS] testUnpauseAndTransfer() (gas: 77323)
Test result: ok. 13 passed; 0 failed; finished in 7.80ms
```

Gas Report

Test result: ok. 14 passed; 0 failed; finished in 8.20ms

contracts/KolectToken.sol:KolectToken contract					
Deployment Cost	Deployment Size				
1695218	8941				
Function Name	min	avg	median	max	# calls
approve	24681	24681	24681	24681	1
balanceOf	605	1405	605	2605	10
burn	4977	10793	12702	14702	3
mint	2857	4234	4768	5079	3
pause	2628	20076	25893	25893	4
transfer	642	15908	15458	32076	4
transferFrom	24612	24612	24612	24612	1
unpause	1594	2122	2122	2650	2

contracts/MockERC20.sol:MockERC20 contract					
Deployment Cost	Deployment Size				
400155	2075				
Function Name	min	avg	median	max	# calls
balanceOf	537	1537	1537	2537	6
transfer	27942	29275	29942	29942	3
transferFrom	8195	8195	8195	8195	1

contracts/MockTVKSwap.sol:MockTVKSwap contract					
Deployment Cost	Deployment Size				
473027	2370				
Function Name	min	avg	median	max	# calls
balanceOfLegacyToken	3259	3259	3259	3259	1
balanceOfNewToken	3204	3204	3204	3204	1
legacyTVKToken	403	403	403	403	10
swapStatus	371	1371	1371	2371	2
swapToggle	2431	6609	8699	8699	3
swapToken	2582	24087	24087	45592	2
tokensInit	44776	44776	44776	44776	10
withdrawToken	2712	17810	17810	32909	2

contracts/VirtuaBinary.sol:NewToken contract					
Deployment Cost	Deployment Size				
1769333	8773				
Function Name	min	avg	median	max	# calls
approve	24681	24681	24681	24681	2
balanceOf	583	1383	583	2583	10
burn	2955	7817	7817	12680	2
burnFrom	13896	13896	13896	13896	1
initialSupply	285	285	285	285	4
maxSupply	283	283	283	283	1
mint	5475	29321	37215	37381	4
pause	23960	24960	24960	25960	2
totalSupply	2394	2394	2394	2394	1
transfer	818	14038	11641	32054	4
transferFrom	2972	2972	2972	2972	1
unpause	1576	1576	1576	1576	1

Concluding Remarks

Building upon the identified findings, Vanar Chain team, in collaboration with ImmuneBytes's auditors, have diligently implemented appropriate resolutions and acknowledgments.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the VANAR Chain platform or its product, nor is this audit investment advice.
Notes:

- Please make sure contracts deployed on the mainnet are the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES

AUDITS